

Python 字符串 — Java 对照速查（完整版）

来源：Day 1 学习记录 + WeChat 9个字符串文件 + 学校实验报告

总览：Python 字符串操作 7 大模块

字符串	
—— 1. 定义与转义	'...' / "..."/ """..."""+ \ ' \ " \ \
—— 2. 格式化输出	% → .format() → f-string（三代演进）
—— 3. 查找与替换	find() / index() / in / replace()
—— 4. 分割与拼接	split() / join() / +
—— 5. 大小写转换	upper() / lower() / capitalize() / title()
—— 6. 空白处理	strip() / lstrip() / rstrip()
—— 7. 对齐与填充	center() / ljust() / rjust()

Java ↔ Python 速查表

维度	Java	Python	备注
类型声明	<code>String s = "hi";</code>	<code>s = "hi"</code>	动态类型
不可变性	✓	✓	完全一致
定义方式	仅双引号	单/双/三引号	Python 更多选择
转义字符	<code>\ " \ ' \ \</code>	完全一致	
格式化	<code>String.format()</code>	<code>f-string</code> （首选） / <code>.format()</code> / <code>%</code>	Python 有三代方案
长度	<code>s.length()</code>	<code>len(s)</code>	⚠ <code>len</code> 是函数不是方法

维度	Java	Python	备注
重复 N 次	需循环	<code>"=" * 40</code>	Python 独有
子串判断	<code>s.contains("a")</code>	<code>"a" in s</code>	和 list 统一用 <code>in</code>
查找位置	<code>s.indexOf("t")</code>	<code>s.find("t")</code>	找不到都返回 -1
查找（指定起点）	<code>s.indexOf("t", 5)</code>	<code>s.find("t", 5)</code>	一致
全部替换	<code>s.replaceAll("a","b")</code>	<code>s.replace("a","b")</code>	
限制次数替换	需手写循环	<code>s.replace("a","b", 2)</code>	Python 更方便
分割	<code>s.split(",")</code>	<code>s.split(",")</code>	完全一致
限制次数分割	<code>s.split(",", 2)</code>	<code>s.split(",", 2)</code>	一致
拼接	<code>String.join("-", list)</code>	<code>"-".join(list)</code>	⚠️ 方向相反！
直接拼接	<code>"a" + "b"</code>	<code>"a" + "b"</code>	完全一致
全大写	<code>s.toUpperCase()</code>	<code>s.upper()</code>	
全小写	<code>s.toLowerCase()</code>	<code>s.lower()</code>	
首字母大写	需手写	<code>s.capitalize()</code>	
每词首字母大写	需手写	<code>s.title()</code>	Python 独有
去两端空白	<code>s.trim()</code>	<code>s.strip()</code>	
去左空白	无直接等价	<code>s.lstrip()</code>	
去右空白	无直接等价	<code>s.rstrip()</code>	
居中对齐	需手写	<code>s.center(13, "-")</code>	Python 内置
左对齐填充	需手写	<code>s.ljust(13, "*")</code>	Python 内置

维度	Java	Python	备注
右对齐填充	需手写	<code>s.rjust(13, "%")</code>	Python 内置
删除指定字符	需正则	<code>s.strip("指定字符")</code>	

一、字符串定义：三种引号

'单引号'

Python 惯例首选

"双引号"

和单引号等价，含单引号时用："let's"

"""三引号

多行文本（Java 15+ 才有文本块）

转义字符（和 Java 完全一样）：

print("let's learn Python")

双引号包裹 → 无需转义

print('let\'s learn Python')

单引号包裹 → 需要 \'

print('How do you spell "Python"?')

单引号包裹 → 双引号无需转义

核心原则：选一种引号做主引号，字符串里出现这种引号时才换另一种或转义。

二、格式化输出：三代演进

第一代：% 格式化（类似 C 语言 printf，不推荐新代码使用）

name = '张三'

age = 27

ave = 88.856

print("姓名: %s" % name)

%s = 字符串

print("年龄: %d岁\n平均成绩%.2f" % (age, ave))

%d = 整数, %.2f = 浮点2位

格式符	含义
%s	字符串
%d	整数
%f	浮点数
%.2f	浮点数保留2位小数

第二代：.format() 方法（三种占位方式）

```
name = '张三'
age = 27

# 方式1: 顺序占位（按 {} 出现顺序）
print("姓名: {}\n年龄: {}".format(name, age))

# 方式2: 编号占位（{0} 是第一个参数，{1} 是第二个）
print("姓名: {1}\n年龄: {0}".format(age, name))    # 故意颠倒了

# 方式3: 名称占位（最清晰，推荐）
print("姓名: {n}\n年龄: {a}".format(n=name, a=age))

# 格式化数字
print('所占百分比: {:.2%}'.format(19/22))        # 86.36%
```

第三代：f-string（Python 3.6+，现代首选）

```
age = 20
ave_score = 88.8534
gender = '男'

print(f"年龄: {age}, 性别: {gender}, 平均成绩: {ave_score:.2f}")
# → 年龄: 20, 性别: 男, 平均成绩: 88.85
```

:.2f 的含义拆解： - **:** — 格式说明开始 - **.2** — 保留 2 位小数 - **f** — float 浮点数格式

三代对比

```
name = "张三"
age = 27
ave = 88.856

# 第一代 % — C 风格，类型靠格式符控制
"姓名: %s, 年龄: %d, 成绩: %.2f" % (name, age, ave)

# 第二代 .format() — 用 {} 占位，类型自动推断
"姓名: {}, 年龄: {}, 成绩: {:.2f}".format(name, age, ave)

# 第三代 f-string — 变量直接写进字符串（推荐！）
f"姓名: {name}, 年龄: {age}, 成绩: {ave:.2f}"
```

日常用 f-string，需要模板复用时用 `.format()`，维护老代码时会看到 `%`。

三、查找与替换

`find()` — 查找子串位置

```
string = "Pythontn"
string.find('t')      # 2 — 从头找，首次出现位置
string.find('t', 5)    # 6 — 从第5位开始找
string.find('z')      # -1 — 找不到返回 -1
```

= Java `indexOf("t")` / `indexOf("t", 5)`，找不到都返回 -1。

Python 也有 `index()` 方法，但找不到会抛异常。日常用 `find()` 更安全。

`in` — 子串存在判断

```
if "python" in "Life is short.I use python":
    print("找到了")
```

= Java `s.contains("python")`，但 Python 的 `in` 对 list 也适用，语法统一。

`replace()` — 替换

```
s = "All things Are difficult before they Are easy Are good"
```

```
s.replace("Are", "are")      # 全部替换 → 3个Are都变are
```

```
s.replace("Are", "are", 2)   # 只替换前2个 → 第三个Are不变
```

⚠ 字符串不可变！`s.replace(...)` 返回新字符串，必须 `s = s.replace(...)` 重新赋值。

四、分割与拼接

`split()` — 分割字符串 → 返回列表

```
s = "The more efforts you make,the more fortune you get"
```

```
s.split()      # 默认按空白字符分割
```

```
# → ['The', 'more', 'efforts', 'you', 'make,the', 'more', 'fortune', 'you', 'get']
```

```
s.split('m')   # 按字母 'm' 分割 (m 本身被丢弃)
```

```
# → ['The ', 'ore efforts you ', 'ake,the ', 'ore fortune you get']
```

```
s.split('e', 2) # 按 'e' 分割，最多分割2次 → 产生3个元素
```

```
# → ['Th', ' mor', ' efforts you make,the more fortune you get']
```

= Java `s.split(" ")`，参数语义完全一致。

`join()` — 拼接列表 → 返回字符串

```
symbol = '*'
```

```
world = 'Python'
```

```
symbol.join(world)  # 'P*y*t*h*o*n' — 在每个字符之间插入 *
```

```
# 实用场景
```

```
"-".join(["2026", "05", "07"])    # "2026-05-07"
"".join(["a", "b", "c"])         # "abc"
```

⚠️ **join 方向记反是高频错误!** 分隔符调 `.join(列表)`，不是列表调。 - ✅ `"-".join(list)` ← 分隔符在前 - ❌ `list.join("-")` ← 报错!

+ — 直接拼接

```
star = 'Py'
end = 'thon'
star + end    # 'Python'
```

五、大小写转换

```
old = 'hello world'

old.upper()      # 'HELLO WORLD'    — 全大写 (= Java toUpperCase())
old.lower()      # 'hello world'    — 全小写 (= Java toLowerCase())
old.capitalize() # 'Hello world'    — 仅首字母大写
old.title()      # 'Hello World'    — 每个单词首字母大写
```

方法	效果	Java 对比
<code>upper()</code>	HELLO WORLD	<code>toUpperCase()</code>
<code>lower()</code>	hello world	<code>toLowerCase()</code>
<code>capitalize()</code>	Hello world	需手写
<code>title()</code>	Hello World	需手写

`capitalize()` vs `title()` 的区别: `capitalize()` 只变第一个单词首字母, `title()` 每个单词首字母都变。

六、空白处理

```
old = ' Life is short,Use Python! '
```

old.strip() # 'Life is short,Use Python!' — 去两端空白 (= Java trim())

old.lstrip() # 'Life is short,Use Python! ' — 去左边空白

old.rstrip() # ' Life is short,Use Python!' — 去右边空白

高级: 去掉指定字符 (Java 没有直接等价)

"...hello...".strip(".") # → "hello"

七、对齐与填充

```
sentence = 'hello world' # 长度 = 11
```

sentence.center(13, '-') # '-hello world-' — 居中, 总长13, - 填充

sentence.ljust(13, '*') # 'hello world**' — 左对齐, * 填充右侧

sentence.rjust(13, '%') # '%hello world' — 右对齐, % 填充左侧

参数含义: s.xxx(width, fillchar) - width — 目标总长度 (如果 ≤ 原字符串长度, 不填充) - fillchar — 填充字符 (必须单字符, 不写默认空格)

常见陷阱 TOP 5

#	错误写法	问题	正确写法
1	s.replace("a","b")	没赋值, 结果被丢弃	s = s.replace("a","b")
2	list.join("-")	join 方向反了	"-".join(list)
3	s.len()	len 是函数不是方法	len(s)
4	s[0] = "A"	字符串不可变	s = "A" + s[1:]
5	str s = "hi"	Python 不声明类型	s = "hi"

全部来源文件索引

文件	模块	知识点
字符串表示.py	定义与转义	单/双/三引号、 <code>\'</code> <code>\"</code> 转义
格式化字符串f-string.py	格式化	f-string + <code>:.2f</code> 精度控制
格式化字符串format方法.py	格式化	<code>.format()</code> 三种占位 + <code>:.2%</code>
格式化字符串%.py	格式化	<code>%s</code> <code>%d</code> <code>%.2f</code>
字符串的查找与替换.py	查找替换	<code>find()</code> + <code>replace()</code> <code>count</code>
字符串的分割与拼接.py	分割拼接	<code>split()</code> 三种 + <code>join()</code> + +
字符串大小写转换.py	大小写	<code>upper/lower/capitalize/title</code>
字符串对齐.py	对齐	<code>center/ljust/rjust</code>
删除字符串的指定字符.py	空白处理	<code>strip/lstrip/rstrip</code>